

[Nom de l'école ou université]

[Filière et option]

Logo de l'établissement

Logo MOGADOR
TECHNOLOGIE
INDUSTRIELLE

RAPPORT DE STAGE

Prevision horaire de la demande electrique

Conception d'une application de prevision et de suivi de qualite des donnees

Realise par :	[Nom et prenom de l'etudiant]
Entreprise d'accueil :	MOGADOR TECHNOLOGIE INDUSTRIELLE
Periode :	[Periode du stage]
Encadrant pedagogique :	[Nom de l'encadrant pedagogique]
Encadrant entreprise :	[Nom de l'encadrant MOGADOR TECHNOLOGIE INDUSTRIELLE]
Annee universitaire :	[Annee universitaire]

*Document a completer avec les informations administratives validees par MOGADOR
TECHNOLOGIE INDUSTRIELLE et l'établissement.*

Remerciements

Je remercie la direction de MOGADOR TECHNOLOGIE INDUSTRIELLE pour l'accueil et l'opportunité de réaliser ce stage. J'adresse mes remerciements particuliers à mon encadrant en entreprise, à mon encadrant pédagogique et à toutes les personnes ayant contribué aux échanges métier et techniques. Leur accompagnement a permis de transformer un besoin de pilotage de la consommation électrique en une solution logicielle structurée et vérifiable.

Resume

Ce rapport presente la conception et la realisation d'une application de prevision horaire de la demande electrique dans le cadre d'un stage chez MOGADOR. La solution combine un modele XGBoost, des variables calendaires, des retards de demande et des donnees meteorologiques. Elle expose ses resultats au moyen d'une API FastAPI et d'une interface React. Le travail ne se limite pas a la prediction : il introduit un contrat d'ingestion pour les donnees reelles, des controles de qualite, une evaluation separee entre prediction a un pas et prevision recursive, des intervalles d'incertitude estimes, ainsi qu'un processus de reentrainement protege.

Les resultats disponibles proviennent du jeu de donnees synthetique inclus dans le projet. Sur la periode de test 2024 tenue a l'ecart, le modele XGBoost actif atteint une WAPE de 3,55% et une RMSE de 130,22 MW. Ces mesures constituent une demonstration technique et ne doivent pas etre interpretees comme des performances operationnelles avant validation sur des donnees reelles de MOGADOR.

Mots-cles : prevision de charge, XGBoost, donnees horaires, FastAPI, React, qualite des donnees, incertitude.

Abstract

This report presents the design and implementation of an hourly electricity-demand forecasting application developed during an internship at MOGADOR. The solution combines an XGBoost model with calendar variables, demand lags and weather data. A FastAPI backend and a React interface make forecasts and evaluation results accessible. The project also establishes a local real-data ingestion contract, data-quality checks, separate one-step and recursive evaluations, estimated uncertainty intervals and a safeguarded retraining workflow.

The available results are based on the project's synthetic dataset. On the untouched 2024 test period, the active XGBoost model reaches a 3.55% WAPE and a 130.22 MW RMSE. These values demonstrate the technical workflow only and require validation on real MOGADOR data before operational use.

Keywords : load forecasting, XGBoost, hourly data, FastAPI, React, data quality, uncertainty.

Table des matières

Remerciements	1
Resume	2
Abstract	3
Liste des abreviations	9
Introduction generale	10
1 Contexte et cahier des charges	11
1.1 Presentation de l'entreprise d'accueil	11
1.2 Contexte metier et problematique	11
1.3 Analyse de l'existant	11
1.4 Expression des besoins	12
1.4.1 Besoins fonctionnels	12
1.4.2 Besoins non fonctionnels	12
2 Etat de l'art et choix technologiques	13
2.1 Prevision de la demande electrique	13
2.2 Technologies retenues	14
3 Conception et architecture	15
3.1 Methodologie de travail	15
3.2 Architecture globale	15
3.3 Pipeline de donnees et caracteristiques	16
3.4 Evaluation et incertitude	16
4 Realisation et implementation	17
4.1 Environnement de developpement	17
4.2 Contrat d'ingestion des donnees reelles	17

4.3	Workflow de reentrainement protege	18
4.4	Tests	18
5	Resultats et discussion	20
5.1	Resultats sur le jeu de demonstration	20
5.2	Limites identifiees	21
	Conclusion generale et perspectives	22
A	Guide d’insertion des captures et diagrammes	24
B	Commandes de verification	25
B.1	Vue generale de l’application	26
B.2	Documentation interactive de l’API	27
B.3	Diagramme des cas d’utilisation	28
B.4	Diagramme de sequence	29
B.5	Diagramme de composants	30
B.6	Diagramme de deploiement local	31
B.7	Dictionnaire de la table historique	32
B.8	Caracteristiques calendaires	33
B.9	Retards et fenetres glissantes	34
B.10	Exemple de fichier de donnees reelles	35
B.11	Exemple de fichier meteorologique futur	36
B.12	Exemple de fichier d’evenements verifiees	37
B.13	Catalogue des controles de qualite	38
B.14	Catalogue des endpoints	39
B.15	Exemple de requete future	40
B.16	Interpretation de la MAE et de la RMSE	41
B.17	Interpretation de la WAPE et du biais	42
B.18	Protocole de validation sur donnees reelles	43
B.19	Procedure de reentrainement manuel	44
B.20	Plan de tests d’integration	45
B.21	Plan de supervision future	46
B.22	Registre des risques de production	47
B.23	Amelioration des intervalles	48
B.24	Etude de modeles futurs	49
B.25	Guide des captures pour la soutenance	50

B.26 Guide de realisation des diagrammes	51
B.27 Glossaire metier et technique	52
B.28 Checklist de finalisation	53
B.29 Matrice de tracabilite des objectifs	54

Table des figures

1.1	Presentation de l'entreprise d'accueil	11
3.1	Architecture fonctionnelle de la solution	15
3.2	Pipeline de traitement et de prevision	16
4.1	Interface de prevision future executee localement	18
4.2	Verification automatisee du projet	19
5.1	Performances du modele, baselines et horizons sur le holdout	21
B.1	Vue generale de l'application locale et avertissement sur les donnees synthetiques	26
B.2	Documentation OpenAPI generee automatiquement par FastAPI	27
B.3	Cas d'utilisation de la plateforme	28
B.4	Sequence de la prevision future	29
B.5	Organisation des composants logiciels	30
B.6	Deploiement actuel de la solution	31

Liste des tableaux

2.1	Comparaison qualitative des approches envisagees	13
5.1	Resultats sur le holdout synthetique 2024	20

Liste des abreviations

API	Application Programming Interface, interface de programmation applicative.
MAE	Mean Absolute Error, erreur absolue moyenne.
RMSE	Root Mean Squared Error, racine de l'erreur quadratique moyenne.
WAPE	Weighted Absolute Percentage Error, erreur absolue ponderee en pourcentage.
XGBoost	Extreme Gradient Boosting, methode d'apprentissage par arbres de decision.

Introduction generale

La consommation électrique varie selon l'heure, le jour, la saison, la météo et les habitudes d'exploitation. Une estimation fiable de la demande permet de mieux planifier les ressources, d'anticiper les pointes et d'appuyer les décisions de pilotage. Dans ce contexte, le stage chez MOGADOR porte sur la mise en place d'un prototype de prévision horaire de la demande électrique, exploitable à travers une interface web locale.

La problématique est la suivante : comment construire une chaîne de prévision qui produise des résultats interprétables, tout en rendant visibles la qualité des données, les limites du modèle et les conditions nécessaires à une utilisation sur des données réelles ? Les objectifs sont de concevoir un modèle de référence robuste, de proposer une API et un tableau de bord, et de préparer le projet à l'intégration future de flux réels de demande et de météo.

Le premier chapitre présente le contexte et les besoins. Le deuxième étudie les choix technologiques. Le troisième décrit l'architecture et le pipeline de données. Le quatrième détaille l'implémentation et les tests. Enfin, le cinquième analyse les résultats, les limites et les perspectives.

Chapitre 1

Contexte et cahier des charges

1.1 Présentation de l'entreprise d'accueil

MOGADOR TECHNOLOGIE INDUSTRIELLE est l'entreprise d'accueil du stage. Cette section doit être complétée à partir d'informations institutionnelles validées : secteur d'activité, organisation, missions, site d'accueil et service de rattachement. Il est important de ne pas ajouter de chiffres, de clients ou de projets non confirmés.

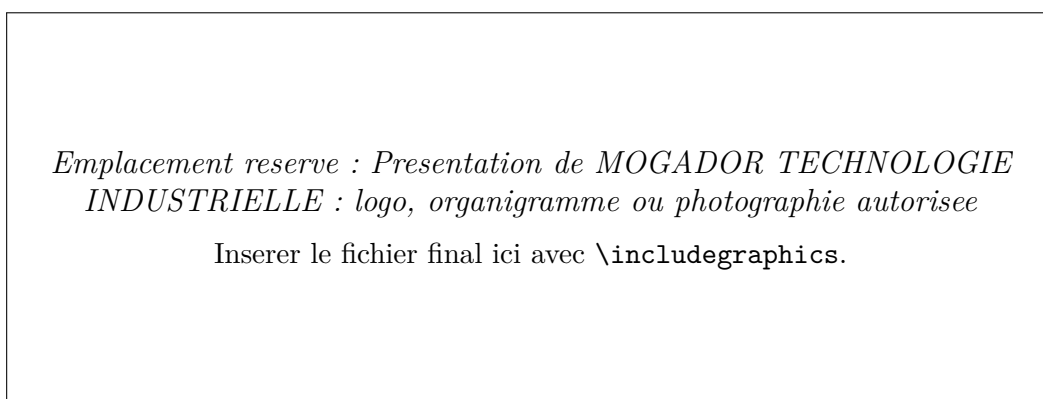


FIGURE 1.1 – Présentation de l'entreprise d'accueil

1.2 Contexte métier et problématique

Le projet traite la prévision de charge électrique à une granularité horaire. Une prédiction calculée à l'avance peut aider à identifier les périodes de forte demande et à mettre à disposition des indicateurs de suivi. Cependant, une prédiction n'est exploitable que si les données utilisées sont identifiées, complètes et compatibles avec le modèle.

1.3 Analyse de l'existant

Le point de départ est un prototype de prévision basé sur un jeu de données local. La version initiale permettait la consultation de données historiques et de prédictions. Les

principaux axes d'amélioration identifiés sont l'explicitation du caractère synthétique des données, la formalisation d'un format d'ingestion, la distinction entre météo observée et météo future, et la mise en évidence des limites de la prévision multi-horizon.

1.4 Expression des besoins

1.4.1 Besoins fonctionnels

- Charger localement des données horaires de demande, température et humidité au format CSV ou XLSX.
- Consulter des statistiques, une analyse de demande, des prédictions historiques et des prévisions futures.
- Visualiser la qualité des données : doublons, trous horaires, valeurs manquantes, valeurs atypiques et couverture temporelle.
- Afficher les performances par horizon et des intervalles de prédiction estimés.
- Conserver les artefacts candidats lors d'un reentraînement et ne promouvoir un modèle que s'il améliore le test tenu à l'écart.

1.4.2 Besoins non fonctionnels

La solution doit être locale, reproductible, testable et explicite sur ses hypothèses. Aucun secret ne doit être codé en dur. Les données réelles et les prévisions météorologiques externes ne doivent pas être inventées ni téléchargées sans configuration explicite.

Chapitre 2

Etat de l'art et choix technologiques

2.1 Prevision de la demande electrique

Les methodes de prevision incluent les approches naives, les modeles statistiques de series temporelles et les methodes d'apprentissage automatique. Dans ce projet, les baselines a 24 heures et 168 heures servent de points de comparaison. Elles reproduisent respectivement la charge du jour precedent et de la semaine precedente.

Les arbres de decision par gradient boosting sont pertinents lorsque les variables explicatives combinent calendrier, meteo et retards de demande. XGBoost a ete retenu car il gere les relations non lineaires, fournit une importance des variables et s'integre simplement a un pipeline Python.

TABLE 2.1 – Comparaison qualitative des approches envisagees

Approche	Atout	Limite	Decision
Naive 24h / 168h	Tres interpretable, rapide	Ne capture pas les variations complexes	Baseline
ARIMA/SARIMA	Base statistique connue	Parametrage delicat avec plusieurs variables	Non retenue pour le prototype
Reseau LSTM	Peut modeliser des sequences longues	Besoin de plus de donnees et de reglages	Non retenue a ce stade
XGBoost	Efficace sur variables tabulaires, interpretable	Prevision recursive sensible a l'accumulation d'erreurs	Retenue

2.2 Technologies retenues

Le backend est developpe avec Python, Pandas, XGBoost et FastAPI. Le frontend utilise React et Vite. Les tests backend sont executes avec Pytest et les tests frontend avec Vitest. Cette separation facilite la reutilisation des fonctions de caracteristiques et d'evaluation entre l'API, les tests et le processus de reentrainement.

Chapitre 3

Conception et architecture

3.1 Methodologie de travail

Le travail est organise en iterations courtes : analyse du comportement existant, ajout d'une amelioration, creation de tests cibles, puis verification du build. Cette approche convient a un prototype logiciel car elle limite les regressions et conserve une trace des decisions techniques.

3.2 Architecture globale

L'application comporte trois couches. La couche donnees lit les fichiers locaux et applique les controles de qualite. La couche metier construit les variables causales, effectue les predictions et calcule les metriques. La couche presentation expose les resultats via FastAPI et React.

*Emplacement reserve : Diagramme d'architecture : fichiers locaux, backend
FastAPI, modele XGBoost, frontend React*
Insérer le fichier final ici avec `\includegraphics`.

FIGURE 3.1 – Architecture fonctionnelle de la solution

3.3 Pipeline de donnees et caracteristiques

Les colonnes requises sont `Timestamp`, `Demand`, `Temperature` et `Humidity`. Les observations doivent etre horaires. Les controles detectent les doublons, les lacunes, les valeurs manquantes et les valeurs atypiques. La meteo peut etre propagee uniquement vers l'avant pour eviter d'utiliser une information future. La demande n'est pas imputee comme cible ; elle est uniquement propagee dans l'historique causal necessaire aux retards.

Les variables incluent l'heure, le jour de semaine, le mois, des encodages cycliques, les jours feriers, la temperature, son carre, des retards de 1 a 336 heures et des statistiques glissantes. Cette construction assure que les variables de demande utilisees pour une heure donnee proviennent strictement du passe.

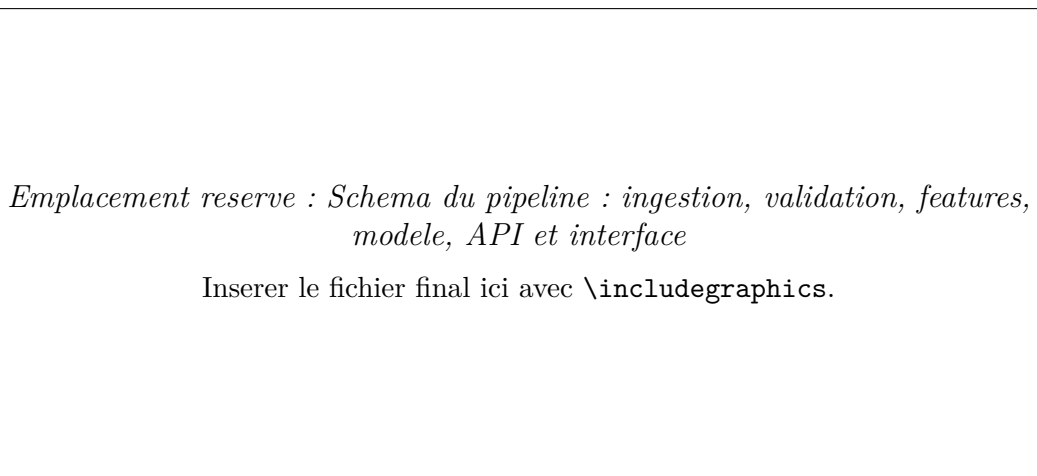


FIGURE 3.2 – Pipeline de traitement et de prevision

3.4 Evaluation et incertitude

Les performances a un pas utilisent les retards et la meteo observees. Les performances multi-pas sont calculees en mode recursif : une prediction peut alimenter les retards suivants. Les deux types de resultats sont presentes separement pour eviter une interpretation abusive.

Un intervalle empirique a 90% est construit a partir des residus absolus du jeu de test. Pour les horizons recursifs, son rayon augmente suivant une regle en racine de l'horizon. Cet intervalle est une estimation et non une garantie de couverture.

Chapitre 4

Realisation et implementation

4.1 Environnement de developpement

Le projet est organise autour des dossiers `backend`, `frontend`, `data` et `models`. FastAPI fournit les endpoints REST ; React affiche les pages Apercu, Analyse de la demande, Performance du modele et Prevision. Les pages de graphiques sont chargees a la demande pour reduire le poids initial du frontend.

4.2 Contrat d'ingestion des donnees reelles

La variable d'environnement `ELECTRICITY_DATA_PATH` indique un fichier local CSV, XLSX ou XLS. La variable `WEATHER_FORECAST_PATH` peut pointer vers un fichier local de previsions meteorologiques contenant `Timestamp`, `Temperature` et `Humidity`. Sans fichier configure, l'application utilise un proxy historique explicitement affiche comme tel. Les evenements non publics sont ajoutes uniquement depuis un fichier verifie, configure avec `VERIFIED_EVENTS_PATH` et contenant `Date` et `Event`.

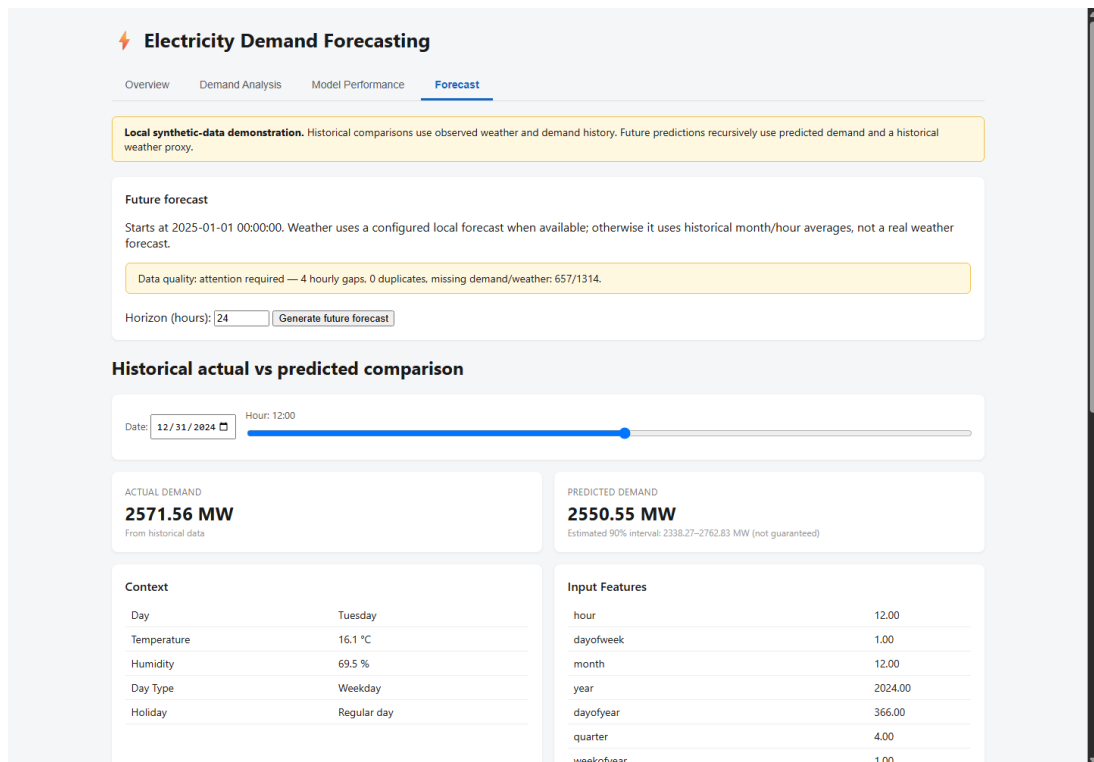


FIGURE 4.1 – Interface de prevision future executee localement

4.3 Workflow de reentrainement protege

Le reentrainement est volontairement manuel. La commande suivante refuse le jeu synthetique integre :

```
python -m backend.modeling -train -data
chemin_vers_donnees_reelles.csv
```

Elle conserve un artefact candidat, emploie des validations chronologiques et compare le candidat au modele actif sur le holdout 2024. La promotion n'est realisee que si la WAPE ou la RMSE s'ameliore strictement sur les memes lignes valides.

4.4 Tests

Les tests verifient les caracteristiques causales, les validations de l'API, la prevision recursive, les metriques, le contrat d'ingestion, la qualite des donnees et l'affichage frontend. La verification finale comprend les tests Pytest, les tests Vitest et la construction du frontend avec Vite.

*Emplacement reserve : Capture du terminal montrant les tests Pytest,
Vitest et le build Vite*

Inserer le fichier final ici avec `\includegraphics`.

FIGURE 4.2 – Verification automatisee du projet

Chapitre 5

Resultats et discussion

5.1 Resultats sur le jeu de demonstration

Le modele XGBoost actif a ete evalue sur une periode de test 2024 non utilisee pendant l'entrainement ni l'arret anticipe. Le tableau 5.1 reprend les indicateurs documentes dans le projet. Les valeurs sont liees au jeu de demonstration synthetique.

TABLE 5.1 – Resultats sur le holdout synthetique 2024

Modele	MAE (MW)	RMSE (MW)	R ²	WAPE (%)
Naive 24 heures	178,25	224,53	0,7860	6,09
Naive 168 heures	144,33	181,99	0,8594	4,93
XGBoost precedent	112,95	141,32	0,9152	3,86
XGBoost actif	103,72	130,22	0,9280	3,55

Ces resultats montrent une amelioration par rapport aux baselines et a l'ancien modele dans le cadre du jeu synthetique. Ils ne valident pas encore une utilisation en production chez MOGADOR. La validation operationnelle necessite des donnees historiques reelles, un referentiel de fuseau horaire confirme et des previsions meteorologiques disponibles au moment de la prediction.

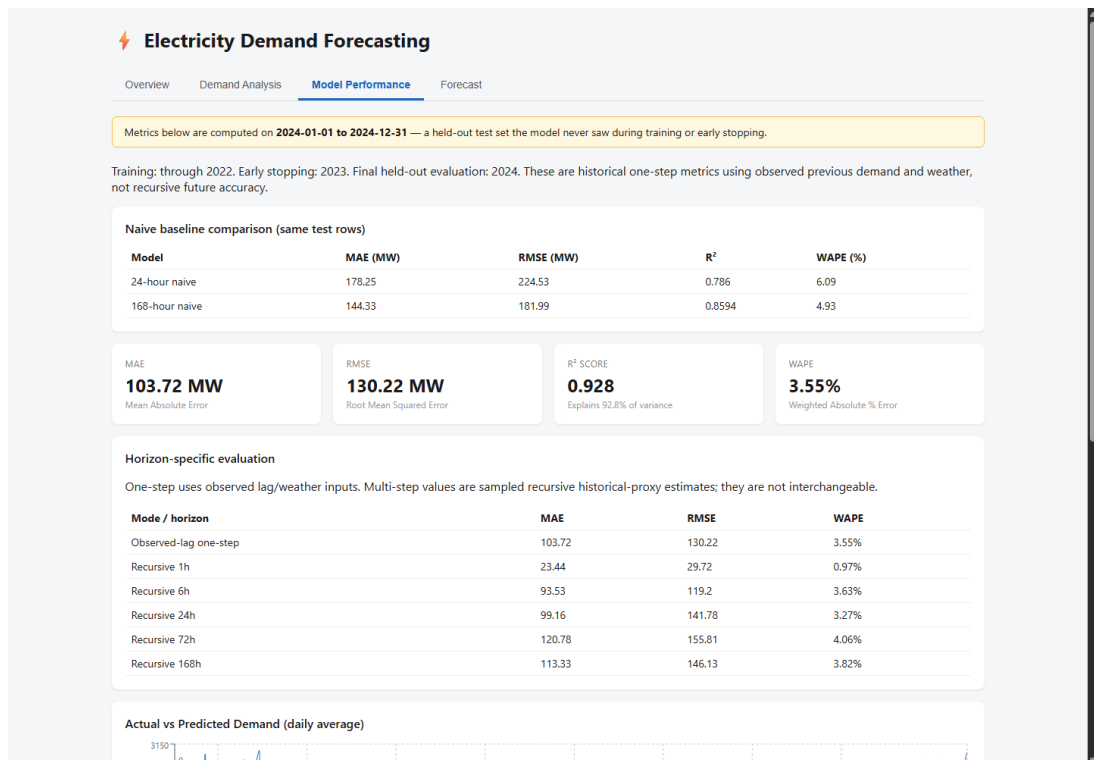


FIGURE 5.1 – Performances du modele, baselines et horizons sur le holdout

5.2 Limites identifiees

La principale limite est l'absence de donnees operationnelles reelles dans le depot. Les valeurs de meteo futures peuvent etre fournies par fichier, mais aucun fournisseur externe n'est branche sans identifiants. Enfin, les previsions recursives accumulent des erreurs a mesure que l'horizon augmente. Les intervalles affiches doivent donc etre lus comme des estimations empiriques et non comme des garanties.

Conclusion generale et perspectives

Le stage a abouti a une application locale de prevision de demande electrique fondee sur XGBoost. La solution fournit une interface web, une API, des metriques de test et des mecanismes qui preparent l'integration de donnees reelles : contrat d'ingestion, controles de qualite, gestion explicite de la meteo future, intervalles estimes et reentrainement protege.

Les prochaines etapes sont l'obtention et la validation de donnees reelles de MOGA-DOR, la definition d'un protocole de mise a jour des donnees, l'integration d'une source meteorologique autorisee, l'ajout de variables metier verifiees et l'evaluation continue des performances par horizon. Une comparaison avec des modeles probabilistes ou des approches de prevision directe par horizon pourra egalement etre etudiee.

Bibliographie

- [1] T. Chen et C. Guestrin, “XGBoost : A Scalable Tree Boosting System”, Proceedings of KDD, 2016.
- [2] FastAPI, “FastAPI Documentation”. <https://fastapi.tiangolo.com/>.
- [3] React, “React Documentation”. <https://react.dev/>.
- [4] Documentation interne du projet, `README.md`, consultee dans le depot local.

Annexe A

Guide d'insertion des captures et diagrammes

Chaque encadre du rapport est une zone reservee. Remplacer, par exemple, un appel a `\visualplaceholder` par :

```
\begin{figure}[H]
  \centering
  \includegraphics[width=0.88\textwidth]{figures/nom-de-la-capture.png}
  \caption{Description precise de la capture}
  \label{fig:nom-capture}
\end{figure}
```

Creer un dossier **figures** a cote de ce fichier $\text{\LaTeX}$ et utiliser des images autorisees par MOGADOR. Pour les diagrammes, privilegier des exports PDF ou PNG lisibles.

Annexe B

Commandes de verification

```
python -m pytest backend/tests -q  
cd frontend  
npm exec vitest run  
npm run build
```

B.1 Vue generale de l'application

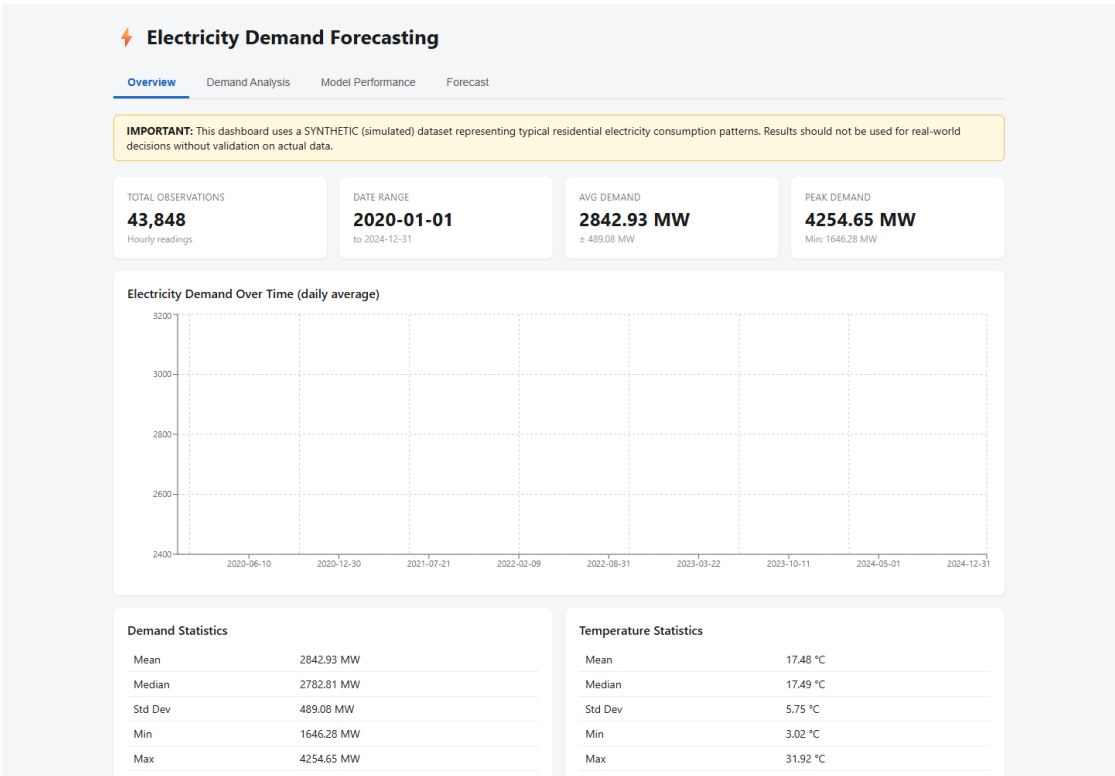


FIGURE B.1 – Vue generale de l'application locale et avertissement sur les donnees synthetiques

La page d'accueil rassemble le nombre d'observations, la plage temporelle, la demande moyenne, le pic et les statistiques meteorologiques. La banniere rappelle que le jeu courant est simule. Cette information doit rester visible avant toute interpretation des graphiques.

B.2 Documentation interactive de l'API

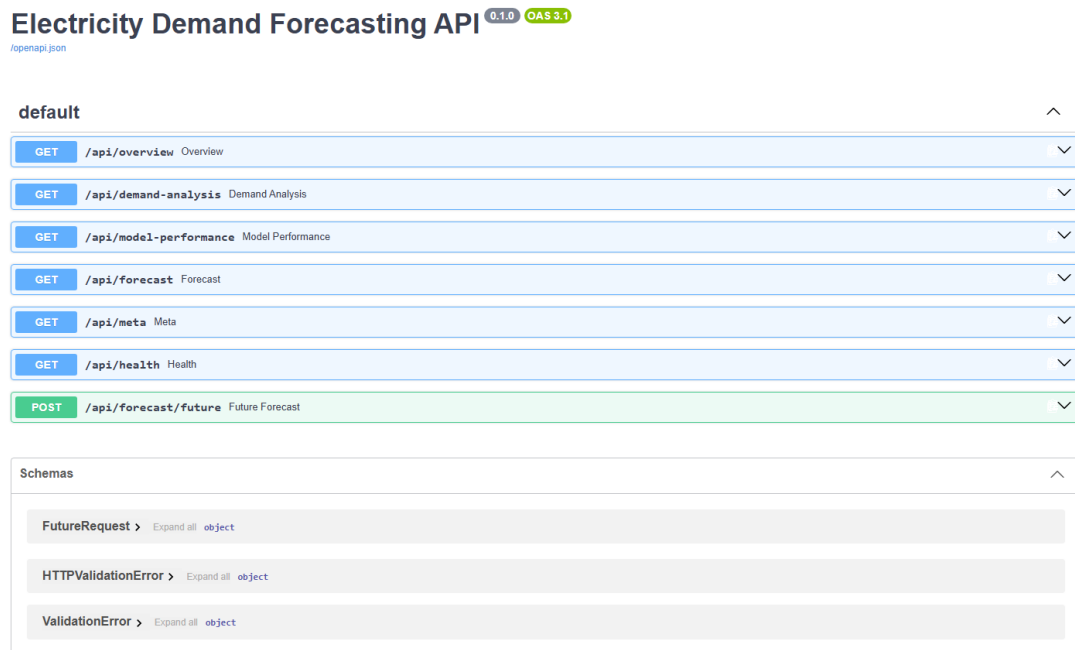


FIGURE B.2 – Documentation OpenAPI generee automatiquement par FastAPI

L'interface Swagger enumere les routes de synthese, d'analyse, de performance, de sante et de prevision. Elle permet de verifier le contrat sans passer par React. Le schema `FutureRequest` documente le debut, l'horizon et les tableaux meteorologiques optionnels.

B.3 Diagramme des cas d'utilisation

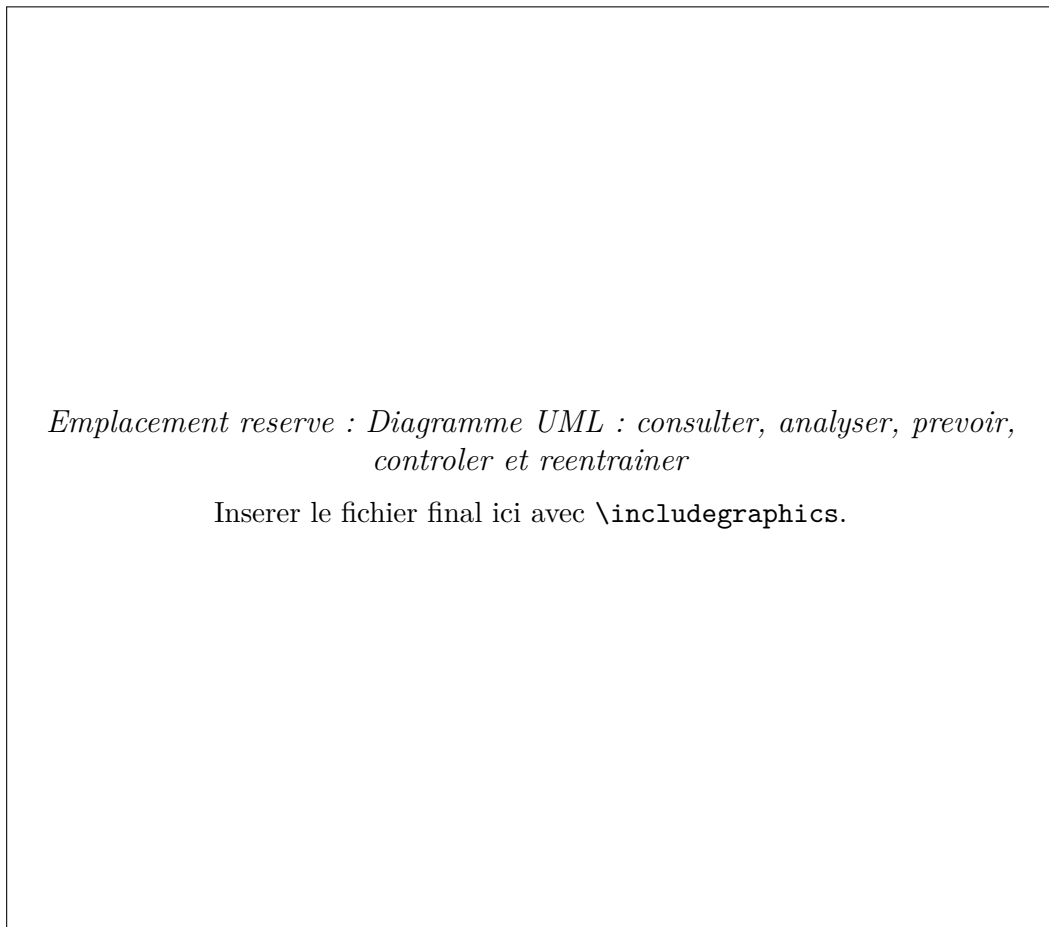


FIGURE B.3 – Cas d'utilisation de la plateforme

Acteurs proposes : utilisateur metier, analyste data et administrateur technique. Re-lie chaque acteur uniquement aux actions effectivement autorisees par MOGADOR TECHNOLOGIE INDUSTRIELLE.

B.4 Diagramme de sequence

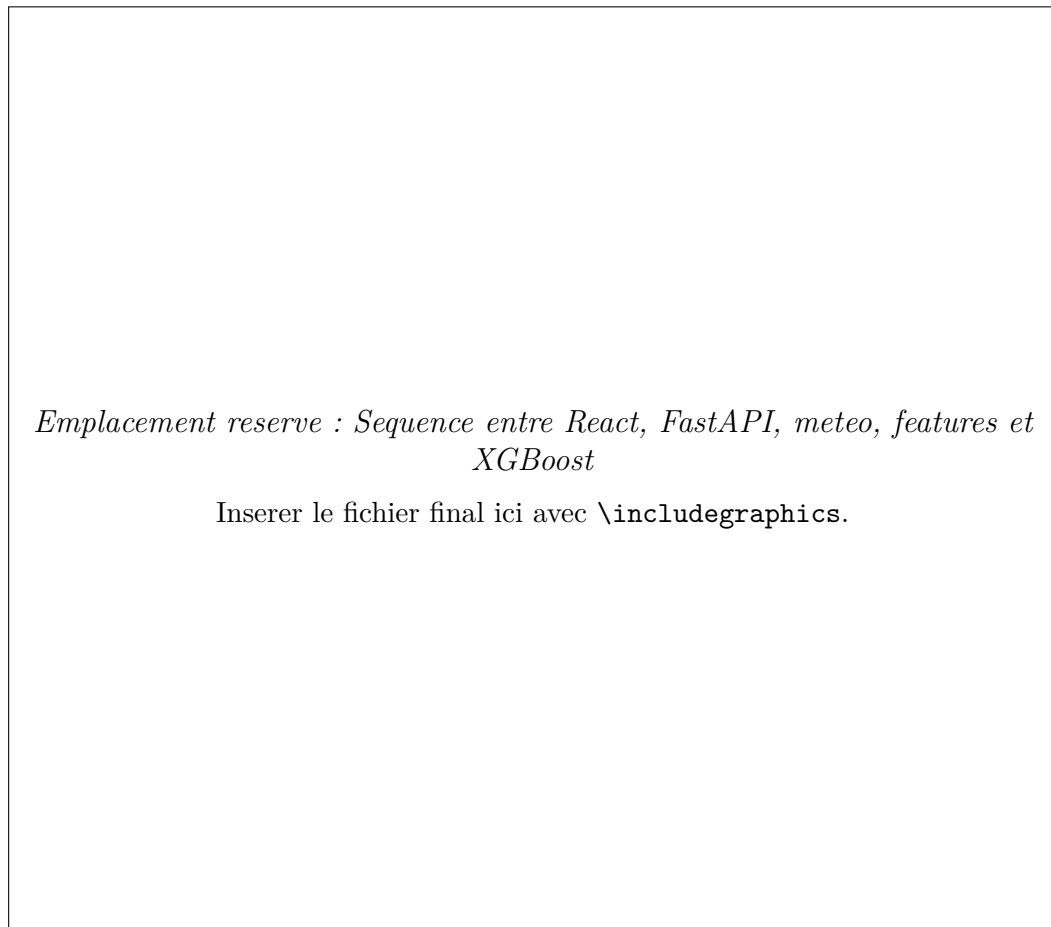


FIGURE B.4 – Sequence de la prevision future

Faire apparaitre la validation de l’horizon, le choix entre meteo fournie et proxy, la boucle recursive et la construction des intervalles. Ajouter un chemin d’erreur pour une couverture meteorologique incomplete.

B.5 Diagramme de composants

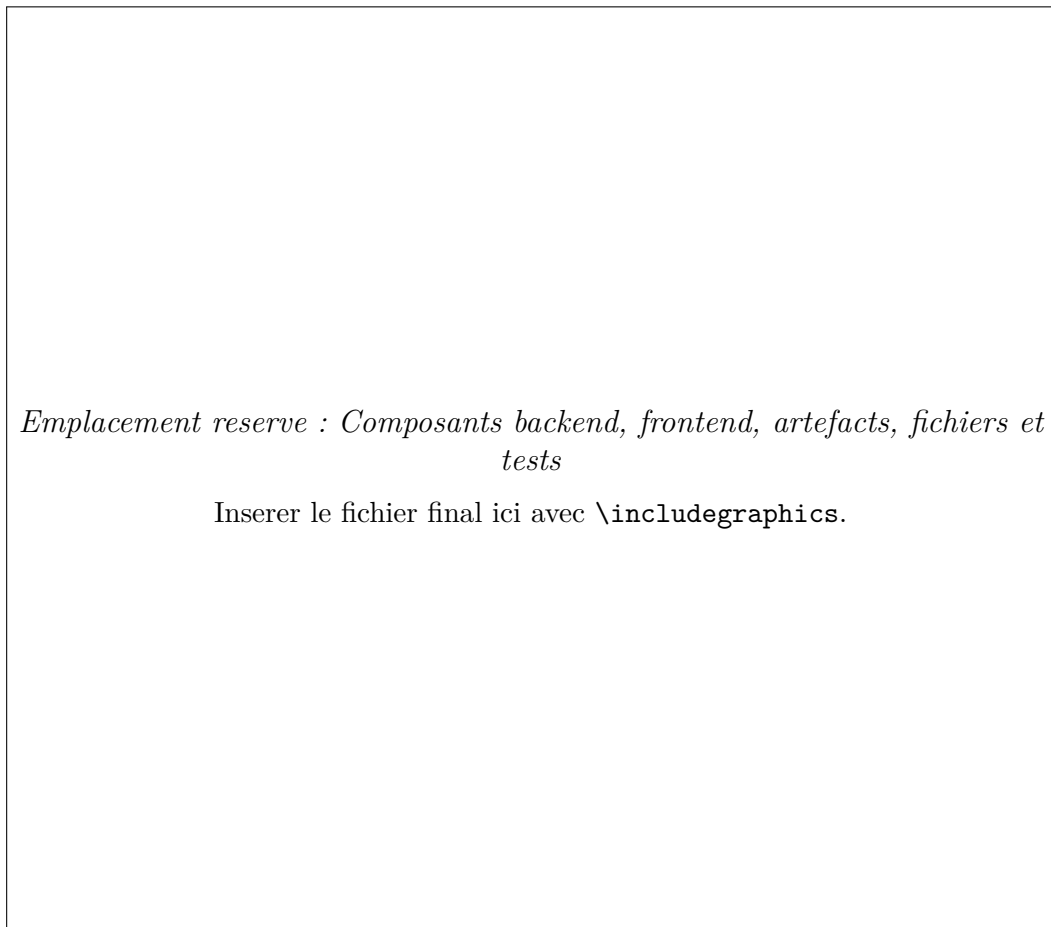


FIGURE B.5 – Organisation des composants logiciels

Le diagramme peut suivre les dossiers du depot. Il doit distinguer les dependances d'execution de celles de developpement et montrer le partage des fonctions de caracteristiques entre entraînement et inference.

B.6 Diagramme de deployment local

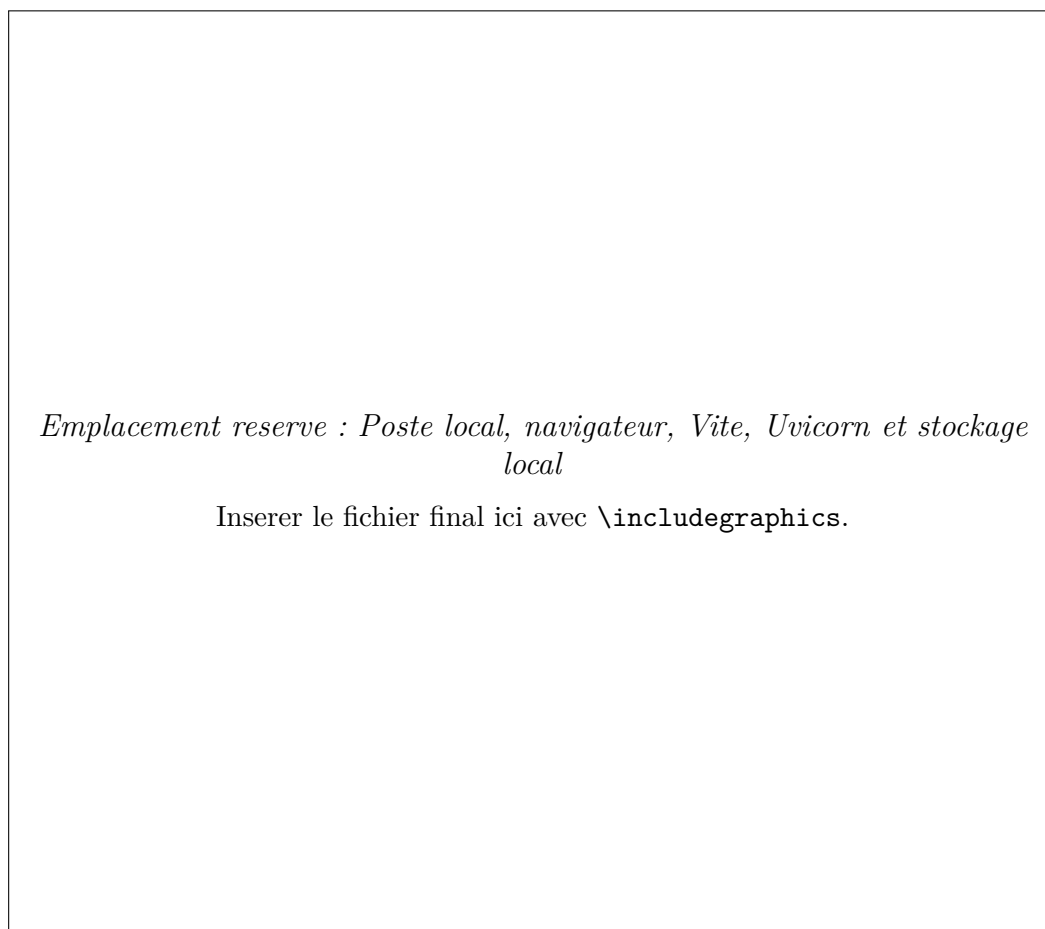


FIGURE B.6 – Deployment actuel de la solution

Le navigateur contacte Vite sur le port 5173 et FastAPI sur le port 8000. Les fichiers de données et de modèle restent sur la machine. Aucune ressource cloud n'est nécessaire dans le perimetre actuel.

B.7 Dictionnaire de la table historique

Champ	Type attendu	Regle
Timestamp	Date-heure	Une valeur par heure, sans doublon
Demand	Numerique	Demande cible en MW
Temperature	Numerique	Unite homogene et documentee
Humidity	Numerique	Pourcentage de 0 a 100

Les colonnes additionnelles sont tolerees mais ignorees par le chargeur canonique. Toute unite differente doit etre convertie avant utilisation et tracee dans la documentation de la source.

B.8 Caractéristiques calendaires

Les variables `hour`, `dayofweek`, `month`, `year`, `dayofyear`, `quarter` et `weekofyear` décrivent la position d'une observation. Les couples sinus/cosinus représentent la continuité entre 23 h et 0 h ou entre dimanche et lundi. Les indicateurs `is_weekend` et `is_holiday` isolent des régimes particuliers. Les jours fériés viennent de la bibliothèque et les événements internes d'un fichier vérifié.

B.9 Retards et fenetres glissantes

Les retards de 1, 2, 3, 6 et 12 heures capturent la dynamique intrajournaliere. Ceux de 24, 48 et 72 heures representent les repetitions quotidiennes. Les retards de 168 et 336 heures correspondent a une et deux semaines. Les moyennes et ecart-types glissants sur 6, 24 et 168 heures mesurent le niveau et la variabilite recents. Toutes ces variables sont calculees apres un decalage causal.

B.10 Exemple de fichier de donnees reelles

```
Timestamp,Demand,Temperature,Humidity  
2026-01-01 00:00:00,<demande>,<temperature>,<humidite>  
2026-01-01 01:00:00,<demande>,<temperature>,<humidite>
```

Cet extrait illustre seulement la structure. Les chevrons doivent etre remplaces par des mesures validees. Le fichier doit etre continu a l'heure, employer un fuseau confirme et conserver des unites constantes. La source, la date d'extraction et le responsable doivent etre documentes.

B.11 Exemple de fichier meteorologique futur

```
Timestamp, Temperature, Humidity  
2026-01-02 00:00:00, <temperature prévue>, <humidite prévue>  
2026-01-02 01:00:00, <temperature prévue>, <humidite prévue>
```

Le fichier doit couvrir chaque heure demandee. Une seule lacune provoque un refus afin d'éviter le melange silencieux d'une vraie prevision et d'un proxy. Le fournisseur, l'heure d'émission et l'horizon du bulletin devront etre traces.

B.12 Exemple de fichier d'évenements verifiés

Date,Event

<date validee>,<nom de l'évenement valide>

Aucune date interne n'est pre-remplie. Les evenements doivent etre connus au moment de la prevision. Un identifiant, une categorie, une heure de debut, une heure de fin et un perimetre pourront completer le schema apres validation metier.

B.13 Catalogue des controles de qualite

Controle	Detection	Traitement
Doublon	Timestamp repete	Statut incompatible
Trou horaire	Grille attendue	Nombre et premier trou
Valeur manquante	Valeur nulle	Comptage par variable
Valeur atypique	Trois IQR	Signalement uniquement
Couverture	Min et max	Exposition API
Historique	Longueur requise	Refus si insuffisant

Une anomalie doit etre examinee avec son contexte. Un seuil ne distingue pas automatiquement une erreur de capteur d'un evenement industriel reel.

B.14 Catalogue des endpoints

Route	Methode	Finalite
/api/overview	GET	Statistiques et serie quotidienne
/api/demand-analysis	GET	Profils et correlations
/api/model-performance	GET	Holdout et horizons
/api/forecast	GET	Comparaison historique
/api/forecast/future	POST	Prevision de 1 a 168 h
/api/meta	GET	Dates et configuration
/api/health	GET	Etat du modele et des donnees

Les erreurs de validation sont exprimees par des codes 4xx. Une erreur serveur doit etre journalisee sans exposer d'information sensible.

B.15 Exemple de requete future

```
POST /api/forecast/future
{"start":"2025-01-01T00:00:00","horizon":24}
```

La date correspond exactement a l'heure qui suit l'historique. Si les tableaux `temperature` et `humidity` sont ajoutes, chacun contient 24 nombres. La reponse identifie la source meteo, le mode recursif, l'avertissement d'horizon et les bornes estimees.

B.16 Interpretation de la MAE et de la RMSE

La MAE conserve l'unité MW et indique l'écart absolu moyen. Une MAE de 103,72 MW signifie que l'erreur absolue est en moyenne de cet ordre sur les lignes évaluées. La RMSE de 130,22 MW est supérieure parce qu'elle accorde davantage de poids aux grandes erreurs. L'écart entre les deux suggère l'existence d'heures plus difficiles. Elles devront être segmentées par saison, plage horaire, type de jour et niveau de charge.

B.17 Interpretation de la WAPE et du biais

La WAPE rapporte l'erreur absolue au volume total de demande. Elle facilite la comparaison entre périodes, mais peut masquer la distribution temporelle. Le biais mesure la direction moyenne. Le biais actif de -28,58 MW indique une sous-prévision sur le holdout. Deux modèles peuvent avoir la même WAPE et des biais opposés ; les deux indicateurs doivent être lus ensemble.

B.18 Protocole de validation sur donnees reelles

Le protocole commence par un audit de la source, du fuseau et des unites. Il reserve une periode finale jamais utilisee pour le choix des variables. Les candidats sont compares sur des validations chronologiques anterieures. Une comparaison finale decide de la promotion. Les resultats sont segmentes par horizon, saison et niveau de charge. Une phase pilote sans impact decisionnel mesure ensuite les ecartes en conditions reelles.

B.19 Procedure de reentrainement manuel

Avant l'exécution, vérifier le chemin, les colonnes, la couverture, les trous et le holdout. Exécuter `python -m backend.modeling -train -data CHEMIN`. Conserver la sortie JSON, le candidat et le rapport. Après une promotion, vérifier l'archive, redémarrer l'API et rejouer les tests. En cas de régression fonctionnelle, restaurer l'artefact archive selon une procédure approuvée.

B.20 Plan de tests d'integration

Le plan couvre le demarrage avec le fichier synthetique, puis avec un fichier valide. Il verifie un fichier absent, une extension refusee, une colonne manquante, un doublon, un trou et une meteo incomplete. Il teste les horizons 1 et 168 et des valeurs invalides. Le frontend doit afficher l'indisponibilite du backend, recuperer apres reessai et conserver les avertissements.

B.21 Plan de supervision future

Une version operationnelle suivra la fraicheur des donnees, le nombre d'heures manquantes, la disponibilite meteo, le temps de reponse, le taux d'erreurs API et la distribution des predictions. Lorsque l'observation devient disponible, le systeme calcule l'erreur par horizon. Les alertes distinguent incident de donnees, panne applicative et derive statistique, avec un responsable et une procedure.

B.22 Registre des risques de production

Risque	Niveau	Reduction
Source non gouvernee	Eleve	Proprietaire et contrat
Fuseau incorrect	Eleve	Tests et validation
Meteo obsolete	Eleve	Controle de fraicheur
Derive	Moyen	Suivi des erreurs
Secret expose	Eleve	Variables et coffre
Modele incompatible	Eleve	Schema versionne

Le risque residuel devra etre estime apres mise en place des controles et approuve par les responsables.

B.23 Amelioration des intervalles

La methode actuelle peut etre remplacee par une regression quantile ou completee par une calibration conforme sur une periode dediee. La couverture sera mesuree pour chaque horizon. La largeur moyenne, la couverture observee et les echecs pendant les pointes sont plus informatifs qu'un pourcentage global. La stabilite saisonniere doit egalement etre testee avant de definir une regle de recalibrage.

B.24 Etude de modeles futurs

Les candidats possibles incluent LightGBM, CatBoost, des modeles lineaires, des modeles statistiques et des architectures temporelles. La comparaison doit employer les memes lignes, horizons et variables disponibles. Une amelioration faible ne justifie pas necessairement une forte complexite. Le protocole notera donc performance, temps d'entrainement, latence, taille, explicabilite et maintenance.

B.25 Guide des captures pour la soutenance

Les captures doivent être refaites après le gel de la version. Utiliser une fenêtre constante, masquer les informations personnelles et conserver les avertissements. Montrer au minimum la vue d'ensemble, la performance, la prévision et l'API. Pour une démonstration réelle, ajouter le statut d'un fichier valide et une prévision avec météo fournie. Chaque image possède une légende et une référence orale.

B.26 Guide de realisation des diagrammes

Le diagramme d'architecture montre les blocs et les flux ; le diagramme de sequence montre l'ordre des appels ; le modele de donnees decrit les entites ; le diagramme de deploiement represente les processus et ports. Eviter de recopier tous les fichiers dans un schema. Chaque figure doit repondre a une question precise et rester lisible une fois imprimee en A4.

B.27 Glossaire metier et technique

Charge : puissance demandee. **Horizon** : distance entre emission et instant predit. **Origine** : debut d'un scenario de backtest. **Proxy meteo** : approximation historique. **Artefact** : fichier contenant modele et metadonnees. **Promotion** : remplacement controle du modele actif. **Derive** : changement de la distribution des donnees ou de la relation avec la cible.

B.28 Checklist de finalisation

- compléter et faire valider les informations administratives ;
- ajouter le logo officiel et la présentation approuvée de l'entreprise ;
- insérer les diagrammes dans les encadres réserves ;
- actualiser les captures après toute modification ;
- modifier les résultats uniquement depuis une évaluation tracée ;
- compiler deux fois et vérifier table, figures et pagination ;
- relire les 55 pages et corriger tout débordement ;
- approuver les informations confidentielles avant diffusion.

B.29 Matrice de tracabilite des objectifs

Objectif	Realisation	Preuve disponible
Ingestion reelle	Contrat CSV/XLSX et validation	Module <code>ingestion.py</code>
Meteo future	Fichier local ou tableaux API	Endpoint de prevision
Evaluation par horizon Incertitude	1, 6, 24, 72 et 168 heures Intervalle empirique a 90%	Page Performance Reponse API et interface
Qualite des donnees	Gaps, doublons, manquants, outliers	Routes Health et Meta
Reentrainement sur	Candidat conserve et promotion garde	Module <code>modeling.py</code>
Evenements verifies	Fichier Date et Event optionnel	Configuration documentee
Resilience frontend	Reessai, avertissements et lazy loading	Tests Vitest

Cette matrice relie le cahier des charges aux composants effectivement realises. Elle facilite la revue avec l'encadrement de MOGADOR TECHNOLOGIE INDUSTRIELLE et permet d'identifier rapidement les preuves techniques a presenter pendant la soutenance. Les elements encore dependants de donnees reelles sont separees des fonctions deja testees sur le jeu synthetique. Lors de la livraison finale, chaque ligne pourra etre completee par un numero de version, une date de validation et le nom du responsable ayant accepte le resultat.